



Framework Para desarrollo de Modelos ML

Table of contents

- [Framework ML 1.0.1](#)
- [Configuration, Installation,](#)
- [Compile and Execution](#)
- [Table of contents](#)
- [Introducción](#)
- [1 Google Cloud Platform](#)
 - [1.1 Project](#)
 - [1.2 Apis](#)
 - [1.3 Cloud Storage](#)
 - [1.4 Iam & Admin](#)
 - [1.5 Bigquery](#)
- [2 Repositorio de Código](#)
 - [2.1 Estructura del repositorio](#)
 - [2.5 Plantilla de Pipeline](#)
- [3 Instalación del Proyecto](#)
 - [Error Reporting](#)
- [ANEXOS](#)
 - [ANEXO - ESTRUCTURA DEL PROYECTO](#)
 - [ANEXO - CODIGO DE DEV A PROD](#)
 - [ANEXO - LINEAMIENTO DE CÓDIGO](#)
 - [ANEXO - EJEMPLO PRICING GCP](#)

Introducción

Los proyectos de Machine Learning (ML) requieren distintas etapas antes de poder llegar a un estado de “Producción”, y un gran porcentaje de estos no suelen pasar de la etapa de “Desarrollo”, debido a la capacidad operativa requerida para lograr configurar, desplegar y mantener un proyecto de ML. Par

Este manual contiene los pasos necesarios para la instalación, preparación y ejecución de un proyecto de ML gestionado por el servicio de GCP - [Vertex Pipeline](#). Previamente un equipo de Advanced Analytics desarrolló y validó procesos (Python, SQL, etc) que cumplieran con una necesidad de negocio. Luego un equipo de operaciones (ML Engineer) ordenará y validará el código, para estructurar los procesos en [componentes](#) de Pipelines, después de un proceso de optimización y configuración la solución completa se almacenará en un repositorio de código (GITLAB). A Continuación mostramos un diagrama detallando las actividades realizadas en el proceso de Instalación, Compilación y Ejecución.

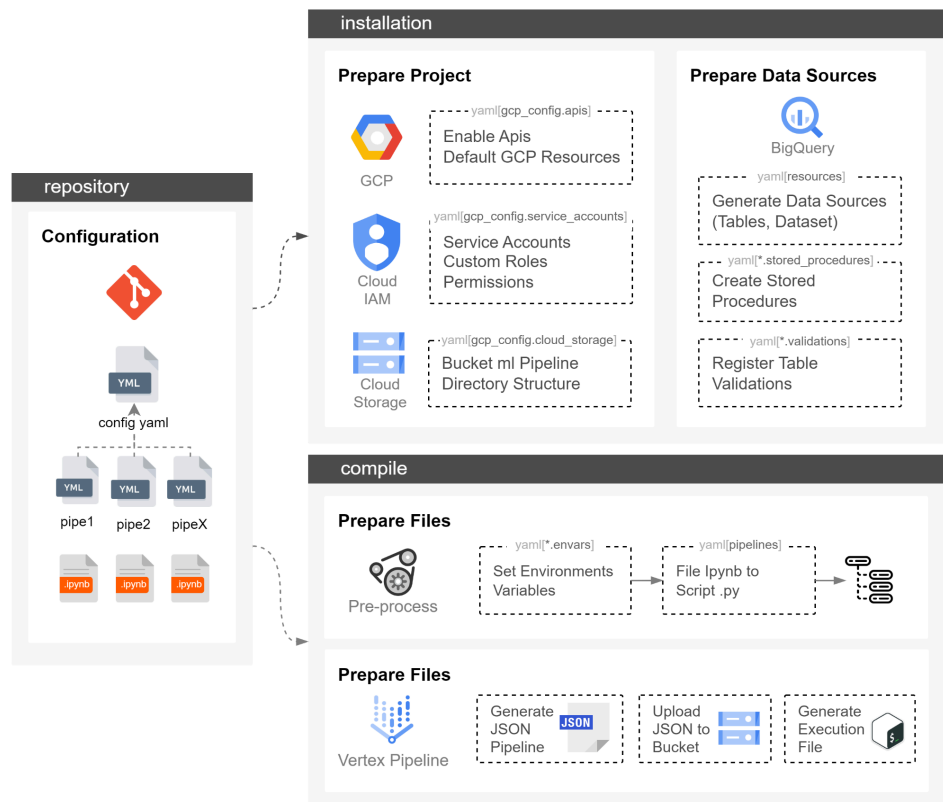


Diagrama de despliegue

1 Google Cloud Platform

1.1 Project

En el documento haremos referencia a algunos datos que serán necesarios para la creación de recursos de GCP

- ❑ **id:** Identificador ÚNICO del caso de uso desarrollado. Por Ejemplo `itc_modelo_recomendacion_als`
- ❑ **project_id:** ID del proyecto de GCP donde se desplegará la solución. Un proyecto GCP puede contener 1 o más casos de uso.
- ❑ **project_number:** Número identificador único asignado a un proyecto de GCP
- ❑ **env** Estado del ambiente donde se desplegará el proyecto (`dev` / `prd`)
- ❑ **company:** Nombre de la empresa o negocio (`intercorp`)
- ❑ **company_abbrev:** Abreviatura del nombre de la empresa o negocio (`itc`)

Labels: Etiquetas gcp para el seguimiento de los proceso ejecutados

- ❑ [application](#): Casos de uso.
- ❑ [module](#): Módulo del caso de uso.
- ❑ [company](#): Nombre de la compañía propietaria del caso de uso.
- ❑ [environment](#): Ambiente de ejecución del caso de uso (dev / prd).
- ❑ [owner](#): Propietario y responsable del caso de uso.
- ❑ [chapter](#): Área de la empresa responsable del proceso

1.2 Apis

Servicios de GCP usados para la ejecución de procesos::

- aiplatform.googleapis.com
- storage-component.googleapis.com
- artifactregistry.googleapis.com
- ml.googleapis.com
- bigquery.googleapis.com
- bigquerystorage.googleapis.com
- compute.googleapis.com
- iamcredentials.googleapis.com

1.3 Cloud Storage

- [bucket_pipeline](#): Bucket Storage donde almacenamos los entregables generados por el pipeline. Ya que los bucket necesitan nombre únicos, debemos generar un valor aleatorio de 6 letras al final del nombre del bucket <random[6]>.

Nomenclatura:

- gs://<env>_<company_abbr>_app_stg_mlops_<random[6]>
- gs://prd_itc_app_stg_mlops_vdsghr

Donde

env: entorno que puede ser “dev” desarrollo o “prd” producción

Company Abbr. Por ejemplo para

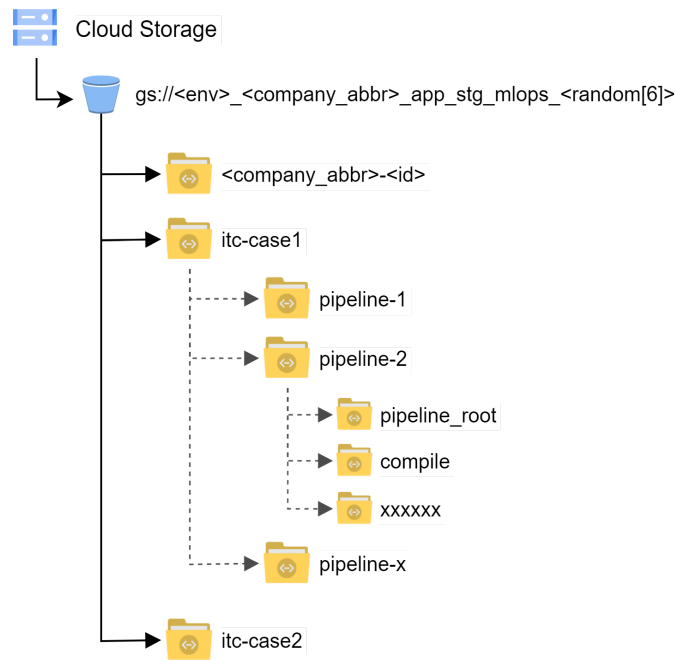
Intercorp: itc,

Farmacias Peruanas: farmas,

Inkafarma: ibk,

Mi farma: mif, etc

Este bucket es provisto para almacenar todos los artefactos y ficheros necesario para la compilación, ejecución y almacenamiento de los casos de uso y sus respectivos pipelines, la estructura básica de carpetas está conformada por 3 niveles de directorios:



Estructura de carpetas

Nivel 1: Carpeta destinada para el caso de uso, pueden existir distintos casos de uso almacenados en un mismo bucket. Nomenclatura `<company_abbr>-<id>`.

Nivel 2: Sub-carpetas destinadas para los pipelines definidos para el caso de uso

Nivel 3: Sub-carpetas destinadas para estructurar los artefactos para el pipeline,

- *pipeline_root*: Ruta proporcionada al compilador en python del pipeline.
- *compile*: Ruta para los pipelines (json) compilados.
- *otras carpetas*, definir las carpetas necesarias para las ejecución del pipeline.

1.4 Iam & Admin

- **roles:** Roles customizados por definir.
- **service_account_process:** Cuenta de Servicio creada para el servicio de Vertex Pipeline, esta será utilizada para la ejecución de los Jobs. Esta cuenta de servicio requiere permisos para conectarse a recursos de Bigquery, Cloud Storage, recursos de datos, entre otros permisos que sean necesarios para el uso de servicios de GCP.

Nomenclatura:

- `<env>-<company_abbr>-mlops-<module>-process@<project_id>.iam.gserviceaccount.com`
- `prd-itc-mlops-nbo-process@<project_id>.iam.gserviceaccount.com`

Permisos gcp:

- `roles/aiplatform.user`
- `roles/artifactregistry.reader`
- `roles/monitoring.viewer`
- `roles/monitoring.metricWriter`
- `roles/logging.logWriter`
- `roles/bigquery.jobUser`
- `roles/bigquery.readSessionUser`
- `roles/bigquery.resourceViewer`
- `roles/iam.serviceAccountOpenIdTokenCreator`

Permisos Bucket Storage:

Esta cuenta de servicio necesita permisos en el [bucket_pipeline](#) para poder consumir y escribir archivos generados en los procesos analíticos. Se necesitan los siguientes permisos.

- `objectViewer`
 - `roles/storage.objectCreator`
 - `roles/storage.objectViewer`
 - `roles/storage.legacyBucketWriter`
-
- **service_account_default:** El servicio de vertex pipelines utiliza por defecto la cuenta de servicio de Compute Engine cuando no se asigna una cuenta de servicio customizada. Por documentación, esta cuenta requiere de ciertos permisos para garantizar la ejecución de los Pipelines

Nomenclatura:

- `<project_number>-compute@developer.gserviceaccount.com`
- Ejemplo `12345678987654-compute@developer.gserviceaccount.com`

Permisos Bucket Storage:

Esta cuenta de servicio necesita permisos en el [bucket_pipeline](#) para poder consumir y escribir archivos generados en los procesos analíticos. Se necesitan los siguientes permisos.

- objectViewer
- roles/storage.objectCreator
- roles/storage.objectViewer
- roles/storage.legacyBucketWriter

- **service_account_execution:** Cuenta de Servicio creada para la ejecución (gatillador) de Jobs de Pipeline.

Nomenclatura:

- `<env>-<company_abbr>-mlps-<module>-execution.iam.gserviceaccount.com`
- `prd-itc-mlps-nbo-execution@<project_id>.iam.gserviceaccount.com`

Permisos GCP:

- roles/aiplatform.user

Permisos Bucket Storage:

Esta cuenta de servicio necesita permisos en el [bucket_pipeline](#) para poder acceder a los pipelines (json) compilados previamente. Se necesitan los siguientes permisos.

- objectViewer
- roles/storage.objectViewer

Permisos a Service Account:

Para que esta cuenta de servicio ejecutora pueda asignar la [service_account_process](#) o [service_account_default](#) al job pipeline, es necesario que la cuenta de procesamiento asigne un permiso de '[roles/iam.serviceAccountUser](#)' a la cuenta ejecutora. Código de referencia:

```
gcloud iam service-accounts add-iam-policy-binding \  
    $[service_account_process | service_account_default] \  
    --member=serviceAccount:$service_account_execution \  
    --role='roles/iam.serviceAccountUser' \  

```



```
--project=$project_id
```

- **google_users:** Entendamos como usuario a una persona que cuenta con un email de Google y que tiene acceso a la consola de GCP (también podría considerarse una cuenta de servicio). Este usuario podría tener 2 funciones principales:
 - Compilar los pipelines
 - Ejecutar los pipelines

Permisos GCP:

- roles/aiplatform.user

Permisos a Google User:

Para que un usuario pueda asignar la [service_account_process](#) o [service_account_default](#) al job pipeline, es necesario que la cuenta de procesamiento asigne un permiso de '[roles/iam.serviceAccountUser](#)' a la cuenta de usuario. Código de referencia:

```
gcloud iam service-accounts add-iam-policy-binding \  
    $[service_account_process | service_account_default] \  
    --member=user:$google_user \  
    --role='roles/iam.serviceAccountUser' \  
    --project=$project_id
```

Permisos Bucket Storage:

Este Google User necesita permisos en el [bucket_pipeline](#) para poder consumir y escribir archivos generados en los procesos analíticos. Se necesitan los siguientes permisos.

- objectViewer
- roles/storage.objectCreator
- roles/storage.objectViewer
- roles/storage.legacyBucketWriter

1.5 Bigquery

Todo proyecto de datos necesita ingestar o retornar información, para ello utilizamos como centro de almacenamiento y procesamiento de datos a Bigquery. Los permisos para recursos de tablas, datasets y Procedimiento Almacenados de bigquery necesitan ser

gestionados de forma manual. Toda nomenclatura para definición de objetos en bigquery ya está de forma general en el documento de estándares

Recursos de datos de ingesta: El acceso a datos deben gestionarse manualmente, para esto es necesario permisos de “[roles/bigquery.dataViewer](#)” o gestionar vistas de bigquery (Opcionalmente Plataforma INCA)

Recursos de procesamiento: Con fines de optimizar el procesamiento de datos optamos por realizar transformaciones con sql en Bigquery, es por eso que gran parte del procesamiento puede gestionarse en queries y/o la creación de Stored Procedures. Recursos necesarios:

- **Dataset:** Se requiere definir un dataset en Bigquery encargado de almacenar los recursos de procesamiento. Es importante definir un dataset asignado a un solo caso de uso (modelo analítico). Por ejemplo, generar un modelo analítico puede requerir distintos stored procedures y tablas intermedias.

Nomenclatura: [<prefix>](#)_model_[<user_area>](#)_[<model_name>](#)

- [prefix](#). Pueden ser
 - [ba](#): Business Analytics
 - [dlv](#): Data Delivery
- [user_area](#): area que utilizara el modelo analitico. Ej: finanzas.
- [model_name](#): nombre de modelo analytics. Eje: recomendación.

Roles: Permisos asignados a la cuenta de servicio de procesamiento

- [roles/bigquery.dataViewer](#)
- [roles/bigquery.dataEditor](#)

- **Stored Procedure:** Para organizar el código sql, puede administrarse a través de procedimientos almacenados.

Nomenclatura: sp_[<company_abrev>](#)_[<group_action>](#)_[<action_name>](#)

- [sp](#): store procedure
- **group_action:** Palabra que define el grupo de acciones o proceso que trabaja el store procedure. Ej: cupones (hace referencia a un proceso de generación de cupones para campañas), segmentacion (haciendo referencia a un proceso de segmentacion)
- [action_name](#): detalle de la acción, utilizar palabras clave

Roles: Los permisos son heredados por el dataset

- **Tablas Intermedias:** Para la ejecución de los procesos y la generación de las tablas de explotación se requieren generar tablas de bigquery que se pueden almacenar temporalmente, hasta que se trunque su valor y que expire la tabla si previamente fue configurado.

Nomenclatura: tmp_<content_name>

- tmp: Temporal
- content_name: Referencia sobre el contenido de los datos

Roles: Los permisos son heredados por el dataset

Recursos productivos: Son los recursos donde almacenamos datos que serán explotados o tablas temporales que serán utilizadas por otros procesos secundarios. Recursos:

- **Dataset Productivo:** Se requiere definir un dataset en Bigquery encargado de almacenar los recursos de datos de explotación. Es importante definir una nomenclatura estándar según lo requiera el caso de uso, se requiere del apoyo de un *Data Enabler*.

Nomenclatura: <bi | ba | etc>_<concept>_<concept_name>

Roles: Permisos asignados a la cuenta de servicio de procesamiento

- roles/bigquery.dataViewer
- roles/bigquery.dataEditor (sólo se requiere para crear recursos nuevos)

- **Tablas Productivas:** Estas requieren ser creadas previamente antes de ser pobladas, se necesita definir su schéma de datos y considerar columnas de gobierno. Algunas veces el proceso que vaya a insertar datos también actualizará o eliminará registros.

Nomenclatura: <bi | ba | etc>_<concept>_<concept_name>

Roles: Permisos asignados a la cuenta de servicio de procesamiento

- roles/bigquery.dataEditor

- **Tablas Auxiliares:** Estas tablas no requieren columnas de gobierno ya que no están destinada para su explotación, pero si se recomienda que se defina su schéma de datos para crear la tabla previamente.

Nomenclatura: aux_<content_name>

- aux: Auxiliar
- content_name: Referencia sobre el contenido de los datos

Roles: Permisos asignados a la cuenta de servicio de procesamiento

- roles/bigquery.dataEditor

2 Repositorio de Código

2.1 Estructura del repositorio

Definida por el framework Dataops

2.5 Plantilla de Pipeline

Dentro del repositorio contamos con una plantilla para ejecutar un “hola mundo” con el servicio de Vertex Pipeline “./workspace/pipeline-demo.ipynb”, este documento se encuentra estructurado de la siguiente manera:

- a. Variables de Entorno del Framework:** Estas variables contienen metadata que fue previamente registrada en el archivo “./config.yaml”, estos datos nos permiten poder configurar información relevante para el despliegue antes de ejecutar la compilación de los Pipelines ([5. Compilación de Pipelines](#)). Estas variables pueden ser estáticas para cada caso de uso. Lista de variables de entorno
- **PIPELINE_CONFIG_ID:** Este campo contiene el identificador único del caso de negocio, el framework lo utilizará para la nomenclatura de algunos recursos generados por los pipelines, también puede utilizarse dentro de la lógica a demanda del desarrollador. El valor proviene del archivo “./config.yaml” en el campo “[config.id](#)”.
 - **PIPELINE_CONFIG_COMPANY:** Contiene el nombre del negocio al que pertenece el caso de negocio. El valor proviene del archivo “./config.yaml” en el campo “[config.company](#)”.
 - **PIPELINE_CONFIG_COMPANY_ABBR:** Contiene el nombre abreviado del negocio al que pertenece el caso de negocio, el valor debe estar alineado con la metadata oficial de las empresas del corporativo. El valor proviene del archivo “./config.yaml” en el campo “[config.company_abbr](#)”.
 - **PIPELINE_DISPLAY_NAME:** Contiene el nombre único del pipeline, es utilizado para ingresar el campo name de sdk kubeflow - código python “@kfp.dsl.pipeline”. El valor proviene del archivo “./config.yaml” en el campo “[pipelines.\[pipeline-X\].config.display_name](#)”.

- **PIPELINE_DESCRIPTION:** Contiene la descripción del pipeline, es utilizado para ingresar el campo description del sdk kubeflow - código python "@kfp.dsl.pipeline". El valor proviene del archivo `"/config.yaml"` en el campo `"pipelines.[pipeline-X].config.description"`.
- **PIPELINE_KEY:** Contiene el identificador único por el cual podemos identificar el pipeline. El valor proviene del archivo `"/config.yaml"` en el campo `"pipelines.[pipeline-X]"`.
- **PIPELINE_COMPILE_FILE:** Contiene la ruta temporal donde se genera el pipeline compilado en formato *.json. Esta ruta se ubica en `"/mlops_template/pipelines/[pipeline-X]"`. Esta ruta es temporal y está excluida del repositorio de gitlab (.gitignore)
- **PIPELINE_SERVICE_ACCOUNT:** Contiene el email de la cuenta de servicio de procesamiento, responsable de ejecutar toda la lógica programada en el Pipeline. Por defecto los componentes ya cargan las credenciales de esta cuenta de servicio, pero se proporciona este campo para los fines que crea conveniente el desarrollador. El valor proviene del archivo `"/config.yaml"` en el campo `"gcp_config.service_accounts.process.name" + "@" + "gcp_config.project_id" + ".iam.gserviceaccount.com"`. O en caso se haya definido el campo `"gcp_config.service_accounts.process.email"`
- **PIPELINE_PROJECT_ID:** Contiene ID del proyecto de GCP donde se desplegarán los pipelines. El valor proviene del archivo `"/config.yaml"` en el campo `"gcp_config.project_id"`.
- **PIPELINE_REGION:** Contiene el nombre de la región del proyecto de GCP donde se desplegarán los pipelines. El valor proviene del archivo `"/config.yaml"` en el campo `"gcp_config.region"`.
- **PIPELINE_BUCKET:** Contiene el nombre único del bucket asignado para almacenar los artefactos de los pipelines. El valor proviene del archivo `"/config.yaml"` en el campo `"gcp_config.cloud_storage.bucket_pipeline.name"`.
- **PIPELINE_LABEL:** Contiene los labels que serán asignados a los recursos y servicios cloud para el seguimiento y monitoreo de procesos. El valor proviene del archivo `"/config.yaml"` en el campo `"gcp_config.label"`.
- **PIPELINE_PATH_ROOT:** Contiene el directorio del Bucket que contendrá todos los artefactos generados en las ejecuciones de cada pipeline. El valor

proviene del archivo `./config.yaml` en el campo `config.company_abbr` + `-` + `config.id` + `pipelines.[pipeline-X]` + `pipeline_root`.

- **PIPELINE_PATH_PROCESS:** Contiene el directorio del Bucket que contendrá todos los recursos de datos que podría requerir un pipeline. El valor proviene del archivo `./config.yaml` en el campo `config.company_abbr` + `-` + `config.id` + `pipelines.[pipeline-X]` + `process`.
 - **PIPELINE_PATH_COMPILE:** Contiene el directorio del Bucket que contendrá el pipeline compilado en formato json. El valor proviene del archivo `./config.yaml` en el campo `config.company_abbr` + `-` + `config.id` + `pipelines.[pipeline-X]` + `compile`.
 - **PIPELINE_RECIPIENTS:** Contiene la lista de emails que recibirán una notificación al finalizar el pipeline. Esto es utilizado solo si el componente de [NotificationsVertex](#) está definido en el código del pipeline. El valor proviene del archivo `./config.yaml` en el campo `config.recipients`.
 - **PIPELINE_VALIDATION_BIGQUERY_TABLES:** Contiene la lista de validaciones de recursos de datos necesarios para ejecutar los pipelines. Por el momento solo se ha desarrollado el validador de Acceso a tablas de Bigquery, Validación de existencia de columnas, y validación de tipos de datos. Posteriormente se seguirá desarrollando este componente. El valor proviene de los archivos config secundarios `./config/config-pipeline-x.yaml` en el campo `validations.bigquery.[*]`.
 - **PIPELINE_COMPONENTS_***: Vertex Pipelines soporta la importación de componentes de vertex previamente compilados, en esta sección iremos agregando componentes reutilizables en los Pipelines. Hasta el momento contamos con 2 componentes:
 - **PIPELINE_COMPONENTS_BQ_VAL_PATH**, componente para validar las tablas registradas en el archivo config.yaml secundario.
 - **PIPELINE_COMPONENTS_BQ_EXPORT_TABLE_TO_GCS**, componente para exportar tablas de bigquery a una ruta de Cloud Storage
- b. Importación de paquetes python:** Importación de librerías requeridas para la construcción del pipeline, las librerías propias de la lógica del proceso no son requeridas ya que esos paquetes se instalan en el componente al momento de ejecutar los pipelines.
- c. Variables de Entorno del Caso de Negocio:** Variables de Entorno definidas en la configuración de los archivos yaml principal y secundarios.

- d. Variables de despliegue Dataops:** Variables propias del framework dataops que forma parte de la configuración que se realiza para adecuar los scripts para el despliegue con el framework Ejemplo \${nombre_variable_despliegue}.
- e. Definición de Funciones y componentes del Pipeline:** Existen 3 tipos de componente de pipelines que se pueden utilizar:
- Componente preconstruídos por google que puede obtener mediante librerías de python
 - Componentes preconstruídos por el usuario, archivos en formato json
 - Componentes creados en el notebook, definidos como funciones
- f. Definición de Función Main Principal que genera el Pipeline:** Definición de la función que crea el pipeline.

```
@kfp.dsl.pipeline(  
    name=PIPELINE_DISPLAY_NAME,  
    description=PIPELINE_DESCRIPTION,  
    pipeline_root=PIPELINE_ROOT  
)  
def main_pipeline()
```

- g. Sentencia para Compilar el Pipeline:** Permite convertir el pipeline construido con el sdk de kubeflow, en un archivo json

```
compiler.Compiler().compile(  
    pipeline_func=main_pipeline,  
    package_path=PIPELINE_COMPILE_FILE  
)
```

- h. Código para Ejecución del Pipeline:** Este segmento de código solo se debe utilizar cuando el pipeline se ejecuta de forma manual ejecutando cada celda del jupyter del pipeline, para la ejecución mediante el framework necesitamos comentar este código

```

import google.cloud.aiplatform as aiplatform
from datetime import datetime

aiplatform.init(project=PIPELINE_PROJECT_ID,
location=PIPELINE_REGION, staging_bucket=BUCKET_NAME)
TIMESTAMP = datetime.now().strftime("%Y%m%d%H%M%S")

job = aiplatform.PipelineJob(
    display_name=PIPELINE_DISPLAY_NAME,
    template_path=PIPELINE_COMPILE_FILE,
    job_id="local-execution-pipeline-{}".format(TIMESTAMP),
    enable_caching=False,
    project=PIPELINE_PROJECT_ID,
    location=PIPELINE_REGION,
    parameter_values={"fecha_proceso": '2021-10',},
    labels=PIPELINE_LABEL,
    failure_policy = 'slow' #'fast' | 'slow'
)

job.submit(service_account=PIPELINE_SERVICE_ACCOUNT)

```

3 Instalación del Proyecto

Toda la instalación se realiza con el framework de despliegue itc dataops

- Creación de objetos como tablas y store procedures
- Creación de pipelines en vertex
- Despliegue de workflows y scheduler para orquestación

[Consideraciones para el despliegue y orquestación:](#)

- a. La invocación del pipeline vertex debe ser realizada vía una cloud function o en algún api de un cloud run.
- b. La orquestación de los pipeline de vertex y stored procedures deben ser realizadas usando workflows
- c. Programación del **"Scheduler"**: Este punto sólo debe ser considerado si el pipeline (job) requiere ser ejecutado periódicamente, para esto puede configurarse un archivo bash que ejecutará el job por línea de comando según lo definido en los puntos anteriores.

En caso de utilizar algún orquestador de procesos (Ejm. Matillion o AirFlow), podemos definir un componente de python y agregar el contenido del archivo "execute.py"

- d. Si el proceso se ejecuta sin problema y el monitoreo del status del proceso es exitoso se genera un comando de salida "exit(0)", que es equivalente a un resultado "SUCCESS", caso contrario "exit(1)" equivalente a "FAILED". Este código de escape del código python de ejecución permite poder capturar el evento de finalización de la ejecución del script, con la finalidad de identificar si el proceso se ejecuta correctamente para así poder ejecutar un siguiente Pipeline del caso de uso.
- e. En caso ocurra un error debemos dirigirnos a la consola de GCP de Vertes Pipeline para ver con detalle los Logs del proceso fallido.

dev-foh-nbprioridad-vtx-push-process-j20221218215547

CLONE STOP DELETE

Runtime Graph 5/8 steps completed Expand Artifacts 80%

Pipeline run analysis

SUMMARY NODE INFO

The replica workerpool0-0 exited with a non-zero status of 1. Termination reason: Error. To find out more about why your job exited please check the logs: https://console.cloud.google.com/logs/viewer?project=624205664083&resource=ml_job%2Fjob_id%2F7618025824112869376&advancedFilter=resource.type%3D%22ml_job%22%0Aresource.labels.job_id%3D%227618025824112869376%22

Execution Info Failed

VIEW JOB VIEW LOGS

Display name	NAME	Type	Duration	Started	Completed
GENERATE BASE PUSH	generatebasepushop	system.ContainerExecution	1 min 43 sec	Dec 18, 2022, 9:56:33 PM	Dec 18, 2022, 9:58:16 PM

Input Parameters

Parameter	Type	Value
bq_project_id	string	int-advanced-analytics-01
drive_directory_id	string	1k9nNEP4dbnbVedBubLP36n-RY8g1ff

dev-foh-nbprioridad-vtx-push-process-j20221218215547

CLONE STOP DELETE

Runtime Graph 5/8 steps completed Expand Artifacts 80%

Logs

MAIN JOB

Logs Severity Default Filter Search all fields and values

SEVERITY	TIMESTAMP	SUMMARY
ERROR	2022-12-18 21:57:59.765 PET	File "/usr/local/lib/python3.7/rumpy.py", line 85, in _run_code
ERROR	2022-12-18 21:57:59.765 PET	exec(code, run_globals)
ERROR	2022-12-18 21:57:59.765 PET	File "/usr/local/lib/python3.7/site-packages/kfp/v2/components/executor_main.py", line 184, in <mo_
ERROR	2022-12-18 21:57:59.765 PET	executor_main()
ERROR	2022-12-18 21:57:59.765 PET	File "/usr/local/lib/python3.7/site-packages/kfp/v2/components/executor_main.py", line 188, in exe_
ERROR	2022-12-18 21:57:59.765 PET	executor.execute()
ERROR	2022-12-18 21:57:59.765 PET	File "/usr/local/lib/python3.7/site-packages/kfp/v2/components/executor.py", line 389, in execute
ERROR	2022-12-18 21:57:59.765 PET	result = self._func(**func_kwargs)
ERROR	2022-12-18 21:57:59.765 PET	File "/tmp/tmp.jieKQ3ptU/ephemeral_component.py", line 186, in generateBasePushOp
ERROR	2022-12-18 21:57:59.765 PET	df_resumen = pd.DataFrame(my_array, columns = ['grupo', 'Total', 'Total_con_token', 'Total_sin_token'...
ERROR	2022-12-18 21:57:59.765 PET	File "/usr/local/lib/python3.7/site-packages/pandas/core/frame.py", line 558, in _init_
ERROR	2022-12-18 21:57:59.765 PET	mgr = init_ndarray(data, index, columns, dtype=dtype, copy=copy)
ERROR	2022-12-18 21:57:59.765 PET	File "/usr/local/lib/python3.7/site-packages/pandas/core/internals/construction.py", line 238, in _
ERROR	2022-12-18 21:57:59.765 PET	return create_block_manager_from_blocks(block_values, [columns, index])

Pipeline run analysis

SUMMARY NODE INFO

The replica workerpool0-0 exited with a non-zero status of 1. Termination reason: Error. To find out more about why your job exited please check the logs: https://console.cloud.google.com/logs/viewer?project=624205664083&resource=ml_job%2Fjob_id%2F7618025824112869376&advancedFilter=resource.type%3D%22ml_job%22%0Aresource.labels.job_id%3D%227618025824112869376%22

Execution Info Failed

VIEW JOB VIEW LOGS

Display name	NAME	Type	Duration	Started	Completed
GENERATE BASE PUSH	generatebasepushop	system.ContainerExecution	1 min 43 sec	Dec 18, 2022, 9:56:33 PM	Dec 18, 2022, 9:58:16 PM

Input Parameters

Parameter	Type	Value
bq_project_id	string	int-advanced-analytics-01
drive_directory_id	string	1k9nNEP4dbnbVedBubLP36n-RY8g1ff
query_base	string	DECLARE _PROCESS_DATE, _DATE; DECLARE COD CANAL . CLIENT TYPE STRING; SET

Error Reporting

- a. **Usuario Supervisor:** Para no sobrecargar al área técnica, se recomienda asignar un responsable de hacer seguimiento del proceso de Vertex Pipeline. Para esto se requiere que al usuario(s) se le asigne los siguientes roles:

- roles/aiplatform.viewer (Vertex AI Viewer)
- roles/logging.viewer (Logs Viewer)
- roles/bigquery.resourceViewer (BigQuery Resource Viewer)

ó para este rol también podemos crear un Custom Role con los siguientes permisos:

- bigquery.jobs.get
- bigquery.jobs.list
- bigquery.jobs.listAll

ANEXOS

ANEXO - ESTRUCTURA DEL PROYECTO

Todo modelo en `service/vertex/[nombre-modelo]/` debe seguir esta estructura:

```
None
[nombre-modelo]/
|
├─ notebook/
|   └─ pipeline-train.ipynb          # Orquesta la compilación
del pipeline de train
```

```
|   └─ pipeline-inference.ipynb      # Orquesta la compilación
del pipeline de inferencia

|

└─ src/

|   └─ __init__.py

|   └─ components.py                # Componentes KFP
(@dsl.component)

|   └─ pipeline_train.py             # Definición del pipeline
de train (@dsl.pipeline)

|   └─ pipeline_inference.py         # Definición del pipeline
de inferencia (@dsl.pipeline)

|

└─ deploy_config_train.yaml          # Config Dataops - train

└─ deploy_config_inference.yaml      # Config Dataops -
inference

└─ requirements.txt                 # Dependencias para
desarrollo local

└─ README.md                       # Definición funcional y
técnica del modelo
```

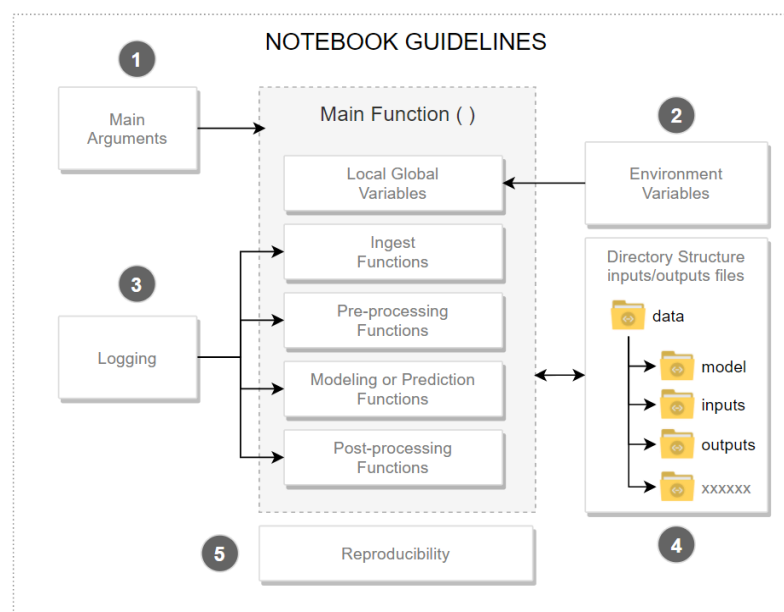
ANEXO - CODIGO DE DEV A PROD

Esto es manejado via variables de despliegue del framework dataops, la idea es no modificar ningun script para desplegar en proyectos de desarrollo o productivos. la variable global que maneja el framework es `${env}` el cual tiene el valor:

dev (Development)

prd (Production)

ANEXO - LINEAMIENTO DE CÓDIGO



- 1. Main Arguments:** Las buenas prácticas recomiendan definir un función principal que abarque toda la funcionalidad de un proceso. En esta función principal podemos definir argumentos para poder comunicarnos con un proceso. Estos argumentos pueden comunicarse al llamar la función o por línea de comando, mediante [Fire](#).
- 2. Environment Variables:** Estas son variables de sistema que pueden ser invocadas desde el código python "**os.getenv**", el framework define y permite definir al usuario variables a demanda.

- ## ANEXO - EJEMPLO PRICING GCP

	SUMMARY	
SERVICE	COST PER YEAR	AVERAGE COST PER MONTH
CLOUD STORAGE	\$0.109	\$0.009
BIGQUERY	\$0.793	\$0.066
CLOUD BUILD	\$0.012	\$0.001
CONTAINER REGISTRY	\$2.520	\$0.210
AI PLATFORM JOBS	\$5.682	\$0.474
TOTAL	\$9.116	\$0.760

[illegible]

query_cost	\$0.0048	0.0084	0.0084	0.0084	0.0084	0.0084	0.0084	0.0084	0.0084	0.0084	0.0084	0.0084	0.0084	0.0084	0.1008
														TOTAL SERVICE	0.793
CLOUD BUILD	COST \ MINUTE	4	0	0	0	0	0	0	0	0	0	0	0	0	
build-minute	\$0.0030	0.0120	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.0120
														TOTAL SERVICE	0.012
CONTAINER REGISTRY	COST \ GB	1.5	1.5	1.5	1.5	1.5	1.5	1.5	1.5	1.5	1.5	1.5	1.5	1.5	
storage	\$0.02	0.030	0.030	0.030	0.030	0.030	0.030	0.030	0.030	0.030	0.030	0.030	0.030	0.030	0.360
red	\$0.12	0.180	0.180	0.180	0.180	0.180	0.180	0.180	0.180	0.180	0.180	0.180	0.180	0.180	2.160
														TOTAL SERVICE	2.520
AI PLATFORM JOBS	COST \ MINUTE	240	240	240	240	240	240	240	240	240	240	240	240	240	
training units	0.2416	0.4735	0.4735	0.4735	0.4735	0.4735	0.4735	0.4735	0.4735	0.4735	0.4735	0.4735	0.4735	0.4735	5.6824
base price	\$0.4900														
														TOTAL SERVICE	5.682
TOTAL PER MONTH		0.728	0.724	0.731	0.739	0.747	0.755	0.763	0.770	0.778	0.786	0.794	0.802		
														TOTAL PER YEAR	9.116

c. Detallado Tablas de BigQuery

			COSTO STORAGE DE 1 GB POR DIA	COSTO INSERT X 1GB	COSTO QUERY X 1GB
			\$0.0007	\$0.0500	\$0.0049
TABLE	GB	DAY S			
TEMPORAL_MOD_REC	0.2163	30	\$0.0043	\$0.0108	\$0.0011
ba_historico_cliente	0.03	30	\$0.0006	\$0.0015	\$0.0001
ba_recomendacion_cliente	0.07	30	\$0.0014	\$0.0035	\$0.0003
ba_top_venta_producto_zona	0.004	30	\$0.0001	\$0.0002	\$0.0000
PROCEDURE_SR	1.4	0	0	0	\$0.0068
TOTAL COSTO X MES			\$0.0064	\$0.0160	\$0.0084

NOTA:

- El detalle de los costos descritos en la tabla anterior, solo contempla los gastos de las ejecuciones programadas. Si el número de ejecuciones se incrementa, influirá directamente en el costo final del proyecto.
- La tabla de costos solo está proyectada a un año de ejecución. Existen algunos gastos (por ejemplo, los de almacenamiento) que son incrementales.
- No estamos considerando los gastos del programador de tareas (Scheduler) del negocio. Recomendamos a la unidad de negocio que para el costo final de la

ejecución del proyecto, considere este, y los gastos que no estén detallados en este documento.